

Aurix Architecture & Execution Workflow

This document outlines the exact execution flow of the Aurix platform, grounded in our architectural plan

([proposed-architecture-details/webDashboard-api-sequence-diagram.mmd](#))

.

Because we are using a modern **Decoupled Microservice Architecture**, each team member can build their component independently. The system relies entirely on passing JSON payloads and interacting via Queues and Webhooks.



1. The Full Execution Loop (How it works)

When a user interacts with the platform, the data flows seamlessly across three entirely separate environments (Frontend, Backend, and the AI Worker Node).

1. User Initiation:

- **Web Dashboard (Bhumika):** A user pastes a GitHub URL into the React dashboard and clicks "Scan".
- **VS Code Extension (Divyanshi):** A developer clicks "Scan Workspace" in their IDE, which securely zips their local code.

2. API Registration (Render Backend):

The Client UI sends the payload (URL or Zip file) to the Backend API built by **Bhavya**. Bhavya's backend creates a "PENDING" scan record in the Supabase database.

3. The Queue Handoff (Upstash Redis):

Bhavya's backend takes the payload and drops it into a Redis Queue (hosted on Upstash) as a "Job Ticket".

4. The Trigger (AWS EC2):

Divyansh's AWS server runs a background loop that constantly checks this Redis Queue every 5 seconds. As soon as it sees a new job in the queue, it pulls it down and automatically starts the 15-minute LangGraph AI scan.

5. The Webhook Return:

Once the AWS server finishes the scan and generates the final JSON report (with all the bugs and AI patches), it automatically POSTs that massive JSON payload back to a specific "Webhook URL" on Bhavya's backend.

6. Database Save (Supabase):

Bhavya's backend catches the JSON payload from the webhook and saves it to the database, updating the scan status to "COMPLETED".

7. Final Display:

- **Web Dashboard (Bhumika):** The React frontend fetches the completed JSON from Bhavya's backend and renders the final vulnerability dashboard.
 - **VS Code Extension (Divyanshi):** The extension fetches the completed JSON, underlines the vulnerable code in the editor, and offers a 1-click Quick Fix patch!
-

2. Why This Architecture is Powerful

Because of this decoupled design:

- **The Client Teams (Bhumika for Web, Divyanshi for VS Code)** never need to talk to the AI engine directly. You only interact with Bhavya's APIs to trigger scans and fetch results.
 - **The Backend Team (Bhavya)** never needs to SSH into the AWS server or run AI scripts. You only need to push jobs into the Redis Queue and build a Webhook URL to catch the results!
 - **The AI Team (Divyansh)** never needs to touch the database. The AWS server simply pulls from Redis and pushes to the Webhook.
-

3. AWS EC2 Worker Node Details (For Reference)

Even though no one except Divyansh needs to log into the AWS server, here are the deployment details for your documentation and architecture presentations:

- **Role:** Dedicated AI Security Worker Node
- **Cloud Provider:** AWS (Amazon Web Services)
- **Instance Type:** `t3.small` (x86_64 architecture)
- **Specs:** 2GB RAM, 30GB SSD
- **Operating System:** Ubuntu 26.04 LTS
- **Public IP Address:** `65.0.134.65`
- **Core Technologies Hosted:** Docker, LangGraph, Groq API, Opengrep, Trivy, Gitleaks, Hadolint.

Next Immediate Step for Backend (Bhavya): Once you set up the Upstash Redis Queue, send the Redis Connection String to Divyansh. He will deploy a 20-line Python script on the AWS server to start listening to your queue, completely automating the pipeline!